Lecture 7 (Part 2): Beyond Linearity + Credit Scoring Project (ISLR Ch. 7)

Polynomials, splines, and a project roadmap

FNCE 5352 — Financial Programming & Modeling (R Weeks 1–7)

2026-01-01

Back from break: where we are

- Part 2: ISLR Ch. 7 — relax the linearity assumption
- Theme: **flexibility knobs** + **CV to choose them**
- Then: credit scoring project walkthrough

Why linearity can fail

- Real relationships often have:
  - thresholds (cliffs)
  - saturation (diminishing returns)
  - asymmetry
- Linear models can miss these patterns

Two strategies for nonlinearity

1. Engineer nonlinear features (polynomials, transforms, interactions)
2. Use models that learn smooth nonlinear functions (splines, GAMs)

Polynomials: the simplest upgrade

- Add $x^2, x^3, \ldots$ to a linear model
- Degree d is a flexibility knob
- Risk: high degrees can wiggle and overfit

Polynomial regression in R (tiny snippet)

- Example: `lm(y ~ poly(x, degree = 3, raw = TRUE), data = df)`
- Or explicitly: `lm(y ~ x + I(x^2) + I(x^3), data=df)`

Why `I()` around `x^2`, `x^3`?

- In R formulas, `^` means "interaction/nesting", not exponentiation
- `I()` ("AsIs") forces the expression to be evaluated literally: `x^2` becomes x^2, not interaction
- Without `I()`: formula would try to fit `x * x` (interaction), not `x` squared

Choosing polynomial degree with CV

Pseudocode: same algorithm, different knob

```
For each degree d = 1, 2, 3, ..., D:
    For each fold i:
        Fit poly(balance, degree=d) on train fold i
        Score on assessment fold i
        Record error
    Average error across folds
Plot CV error vs degree
Pick degree with lowest error
```

- Same mental model as model size / lambda tuning
- Degree is the “flexibility knob”

Polynomials can behave badly at the edges

Interpolation vs. extrapolation:

- High-degree polynomials fit beautifully *within* the domain of your training data
- But *outside* that domain, they can explode or oscillate wildly
- This is a fundamental limitation: you're fitting a function with no anchor beyond the edges

Why extrapolation matters in finance

Stress testing & scenario analysis:

- Your training data covers a range of market conditions (e.g., credit spreads 50–200 bps)
- But stress testing, scenario analysis, and regulatory models often *extrapolate*: "What if spreads hit 500 bps?"
- A high-degree polynomial trained on historical data can give you nonsense in stressed scenarios
- This is a *feature risk*: your model works great on familiar terrain, but fails when the market moves into new territory

The fix: splines or domain constraints

Splines (next): piecewise low-degree polynomials have stable extrapolation behavior

Or domain knowledge: explicit bounds, simpler functional form for tails, regularization toward economic intuition

Splines: piecewise polynomials with continuity

- Idea: use low-degree polynomials in regions
- Join them smoothly at “knots”
- Flexibility controlled by number of degrees of freedom / knots

Basis functions: how splines fit in linear regression

The trick: convert nonlinearity into a linear problem

$$y = \beta_0 + \beta_1 b_1(x) + \cdots + \beta_m b_m(x) + \varepsilon$$

- $b_j(x)$ are *basis functions*: pre-computed transformations of x
- For splines: each basis function is a "bump" or polynomial piece
- We fit *linear* coefficients β_j to these transformations
- Nonlinearity comes from the basis functions, not the fitting algorithm

Why this matters:

- We can reuse everything: `lm()`, `glm()`, `coef()`, prediction intervals, inference
- We just pre-process x first

Splines in R (tiny snippet)

- Use `splines::bs(x, df = 6)` to build a spline basis
- Then: `lm(y ~ bs(x, df=6), data=df)`

Natural splines: what's different?

Regular splines (B-splines) vs. natural splines

- Regular splines are flexible but can behave oddly at the boundaries
- *Natural* splines add boundary constraints: force the function to be *linear* beyond the boundaries
- This means: outside your training domain, it extrapolates linearly (stable, sensible)

Natural splines in finance (and in R)

Why this matters: extrapolation

- In our stress-testing context: natural splines won't explode at extreme spreads
- Trade-off: less flexibility at boundaries, but stable extrapolation behavior

In R:

- `splines::ns(x, df = ...)` (same API as `bs()`, but with boundary constraints)

GAMs: generalized additive models (conceptual)

Idea: sum of smooth functions (one per predictor)

$$y = \beta_0 + f_1(x_1) + f_2(x_2) + \cdots + f_p(x_p) + \varepsilon$$

- Each f_j is a smooth (spline, kernel, etc.); fitted automatically
- Interpretable: plot each smooth effect independently (not coefficients, but *functions*)
- Great for credit risk: nonlinear relationships but you still need to explain them to regulators

How it differs from polynomial/spline regression:

- You fit separate smooths for each predictor (not a global polynomial)
- Captures interactions *implicitly* (each smooth can adapt to regions)
- Needs tuning: smoothness (or degrees of freedom) per predictor

In R: `mgcv::gam(y ~ s(x1) + s(x2), family=binomial(link='logit'))`

Minimal GAM in R (tiny snippet)

- `library(mgcv)`
- `gam(y ~ s(x1) + s(x2) + x3, data=df, family=binomial())`

Selection again: every method has a knob

- Polynomials: degree d
- Splines: df / knots
- GAMs: smoothness / basis dimension
- Regularization: λ
- We choose knobs with CV (and keep a final test set)

How to read a spline model output

- The coefficients are on basis functions (not directly on x)
- Interpretation comes from plotting the fitted curve
- So: focus on *shape*, not individual betas

Knots (conceptual) — where flexibility lives

- Knots are locations where the piecewise polynomials join
- More knots ⇒ more local flexibility (better fit in some regions)

Two ways to control knots:

- **By hand**: specify exact locations: `bs(x, knots=c(25, 50, 75))` → you must decide where
- **By degrees of freedom**: let R space them for you: `bs(x, df=5)` → R picks knot placement

Why use `df`?

- Simpler: "How much flexibility?" vs. "Where should knots be?"
- CV tunes the flexibility level; R handles knot placement automatically

Nonlinearity for classification: same tools

- Replace `lm()` with `glm(..., family=binomial())`
- Polynomials/splines/GAMs still work
- You still need CV + a test set

Model evaluation reminders (for the project)

- **AUC:** ranking quality (threshold-agnostic); good for class imbalance
- **Log loss (cross-entropy):** probability quality (penalizes overconfidence)
- **Confusion matrix metrics:** precision, recall, F1 depend on a threshold choice
- **In credit:** threshold often comes from costs/capacity; AUC and log-loss are safer defaults

Class imbalance (credit reality check)

- Defaults are usually rare
- Accuracy can be misleading (predict “no default” and look great)
- Prefer AUC / PR-AUC / cost-based metrics; consider weighting

Quick aside: time series / rolling evaluation

- If data are ordered in time:
- random folds can leak the future into the past
- Use rolling / expanding windows instead

From methods to project: what levers you now have

- Baseline: logistic regression (linear log-odds)
- Levers:
  - regularization (ridge/lasso)
  - nonlinear terms (poly/splines/GAM)
  - feature engineering (transformations/interactions/missingness)

Credit scoring project: purpose

- You'll build a model that predicts default risk
- Goal: strong out-of-sample performance
- Constraints: avoid leakage; keep work reproducible

Data splits: roles (non-negotiable)

- **Train:** fit models and do CV/tuning
- **CV folds (inside Train):** compare models + tune knobs
- **Test:** final, once-only evaluation (no tuning)
- **Leakage example:** if you pick features on full data, then CV it → test error looks too good

Deliverables (typical pattern)

1. A reproducible R script / notebook that trains + validates
2. A short write-up: what you tried and why
3. A submission file with IDs + predicted probabilities (if required)

Baseline model: logistic regression

- Start with a simple GLM (binomial logit)
- Pick a reasonable set of predictors
- Establish a baseline metric on CV + test

Metric choice (tie to Lecture 5)

- If costs are symmetric: AUC can be a good ranking metric
- If you will threshold decisions: consider cost-weighted error
- Always report something interpretable

Improvement lever 1: regularization (ridge/lasso)

- Use `glmnet` with `family='binomial'`
- Tune λ via CV
- Compare λ_{min} vs λ_{1se}

Improvement lever 2: nonlinear terms

- Try polynomial terms (degree 2–3) for key variables one at a time
- Try spline bases (df = 3–6) for smoother, more local effects
- Tune degree / df with CV inside training set
- Finance use: balance vs default may be threshold-like; utilization may saturate

Improvement lever 3: feature engineering ideas

- Missingness indicators (is.na flags)
- Transforms: log(1+x), winsorization ideas
- Interactions: e.g., income × utilization
- Scaling (when needed) done *inside* resampling

Project hygiene checklist

- Set a seed; save splits; avoid re-splitting every run
- Keep a clear train/CV/test separation
- Log experiments: what changed, what improved
- Sanity-check predictions (calibration-ish, extreme probs)

Starter code walkthrough (what to look for)

- Where the data live (folder path)
- Where you define your split + folds
- Where you compute metrics (yardstick allowed)
- Where you write predictions to CSV

Suggested workflow for the next week

1. Run baseline end-to-end
2. Add one improvement lever at a time
3. Compare using the same CV folds
4. Freeze your final model; score test once; submit

What success looks like (end-to-end project summary)

1. **Baseline:** simple logistic GLM with standard predictors (establish benchmark)
2. **Experiment:** try regularization, nonlinear terms, feature engineering separately
3. **Validate:** report train/CV/test metrics consistently; show reproducible seed/splits
4. **Document:** brief write-up explaining what you tried and why it worked (or didn't)
5. **Submit:** predictions on test set (or specified format) with clear reproducibility

PASTE SCREENSHOT HERE — ISLR Figure suggestions

- Ch. 7: polynomial fit vs spline fit illustration
- Ch. 7: GAM component plots (if you want a visual)
- Ch. 7: CV for selecting flexibility (degree/df)

(Add page numbers once you have your copy open.)